

Project Executable Files

Date	1 st July ,2026
Team ID	xxxxxx
Project Name	Credit card approval prediction
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	YES
2	README / Setup Guide (instructions to run the project)	YES
3	requirements.txt / package.json / dependency file	YES
4	Database schema / seed files (if applicable)	YES
5	Environment configuration file (.env.example or similar)	YES
6	Deployed application URL (if hosted)	YES
7	APK / Executable binary (if applicable for mobile/desktop apps)	YES
8	Dockerfile / Containerization config (if applicable)	YES
9	Test files and test results	YES

10	Demo video or walkthrough (if required)	YES
----	---	-----

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

[Paste or describe your folder structure here. Example:]

```

/project-root
/src    - Source code files
/tests  - Test files
/docs   - Documentation
/config - Configuration files
README.md - Setup and run instructions
requirements.txt - Python dependencies (or package.json for Node)
  
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://github.com/swathipraneethanukala/credit-card-approval-prediction
Login Credentials (Demo)	[Provide demo username and password for evaluator access]
Platform / Hosting Provider	[e.g., Render, Vercel, AWS, Heroku, Azure]
Repository Link	https://github.com/swathipraneethanukala/credit-card-approval-prediction
Demo Video Link	https://youtu.be/[credit-card-approval-prediction

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Pre-requisites:

[List required software, versions, and system requirements]

Step 1: [e.g., Clone the repository: `git clone <url>`]

Step 2: [e.g., Install dependencies: `pip install -r requirements.txt`] Step

3: [e.g., Configure environment: copy `.env.example` to `.env`] Step 4:

[e.g., Run database migrations: `python manage.py migrate`] Step 5:

[e.g., Start the application: `python manage.py runserver`] Step 6:

[e.g., Open browser at `http://localhost:8000`]

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Development Server Performance Warning: Flask development server defaults to single-threaded mode and may lag when handling heavy simultaneous user loads.	Resolved: Configured Dockerfile to launch through gunicorn (WSGI web production wrapper) for parallel request handling in active hosting environments
2	Sparsely Populated Data Fields: The raw dataset features high proportions (30.5%) of missing records under the <code>OCCUPATION_TYPE</code> parameter.	Resolved: We implemented an automated cleaning handler in <code>train_model.py</code> to tag empty records with Unknown string identifiers, preserving rows without compromising data accuracy.

3	Model Retraining Latency: Re-running the baseline training file takes ~45 seconds due to joining operations across millions of credit transaction arrays.	Status: By saving fully compiled pipeline objects directly into static model.pkl and scaler.pkl files, web request loops run in under 0.15s without needing back-end database querying.